Day 142 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 21, Day 3 reference

Watch: Generative Models 101: VAEs, GANs, Autoregressive & Diffusion Models Explained (CorestratAI) | Denoising Diffusion Implicit Models (DDIM) Explained (ExplainingAI)

`day142_generative_tradeoffs.py`, segment by segment, in the order `main()` uses things. The averaged copy of the weights, DDIM, the x_0 guesser wrapper, the grey share and the weight count are new; the VAE (Day 129), prior sampler (Day 130), GAN step (Day 134), scorer (Day 138), schedule and sampler (Day 140) and diffusion model (Day 141) are imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: t, s = steps, s earlier than t · A_t = ABAR[t] · η = eta, the fresh-noise setting · d = DECAY = 0.999

Segment 0 — Setup

Imports: the thread setting comes before numpy

CODE

```
os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")
import contextlib
import io
import sys
import time
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
for w in ("week-20-generative-models-gans", "week-19-unsupervised-learning-ii",
    "week-18-unsupervised-learning-i", "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_exercise as d126
    import day126_autoencoders as d126m
    import day129_exercise as d129
    import day130_exercise as d130
    import day134_exercise as d134
    import day139_gan_build_and_forward_diffusion as d139
    import day140_exercise as d140
    import day141_exercise as d141
```

- `os.environ.setdefault("OPENBLAS_NUM_THREADS", "4")` — sets the variable only if it is not set already; it must run before numpy is imported, which reads it once.
- `import day139_gan_build_and_forward_diffusion as d139` — a lesson script imported as a module: only its functions are used; its experiments sit under `if __name__ == "__main__"` and do not run.
- `import day141_exercise as d141` — importing an exercise runs its checks once; redirect_stdout hides what they print.

Constants

CODE

```
ABAR = d140.ABAR
STEPS = 30000
DECAY = 0.999
CHECKS = (5000, 10000, 20000, 30000)
GAN_EPOCHS = (1, 5, 10, 20, 30)
VAE_EPOCHS = 30
```

- `DECAY = 0.999` — the averaged copy keeps 99.9% of itself at each step.
- `CHECKS = (5000, 10000, 20000, 30000)` — a tuple tested with `s in CHECKS`.

Segment 1 — The new parts

ema_update: changing arrays in place

MATH

$$W_{avg} \leftarrow d\, W_{avg} + (1 - d)\, W$$

CODE

```
def ema_update(ema, net, decay=DECAY):
    for (We, be), (W, b) in zip(ema, net):
        We *= decay; We += (1 - decay) * W
        be *= decay; be += (1 - decay) * b
```

- `for (We, be), (W, b) in zip(ema, net):` — zip pairs layer k of the copy with layer k of the network; each pair is unpacked into two (weights, bias) tuples.
- `We *= decay; We += (1 - decay) * W` — *= and += change the array itself, so the copy stored in `ema` changes; `We = decay * We` would only rebind the name.
- `(no return)` — the function returns None: its whole effect is on the arrays it was given.

ddim_sample: jumps instead of steps

MATH

$$x_s = \sqrt{A_s}\, x'_0 + \sqrt{(1 - A_s - \sigma^2)}\, \varepsilon' + \sigma\, z, \quad \sigma^2 = \eta^2\, (1 - A_s)/(1 - A_t)\, (1 - A_t/A_s)$$

CODE

```
def ddim_sample(x0_fn, n, rng, steps, eta=0.0, d=784):
    ts = np.linspace(len(ABAR) - 1, 0, steps + 1).round().astype(int)
    x = rng.standard_normal((n, d))
    for t, s in zip(ts[:-1], ts[1:]):
        x0 = x0_fn(x, t); e = (x - np.sqrt(ABAR[t]) * x0) / np.sqrt(1 - ABAR[t])
        sig = eta * np.sqrt((1 - ABAR[s]) / (1 - ABAR[t]) * (1 - ABAR[t] / ABAR[s]))
        x = np.sqrt(ABAR[s]) * x0 + np.sqrt(np.maximum(1 - ABAR[s] - sig ** 2, 0)) * e
        if s > 0:
            x = x + sig * rng.standard_normal(x.shape)
    return x
```

- `np.linspace(len(ABAR) - 1, 0, steps + 1).round().astype(int)` — steps + 1 evenly spaced values from 1,000 to 0, rounded to whole step numbers.
- `for t, s in zip(ts[:-1], ts[1:]):` — two slices of the same array, one shifted by one: each pair is a jump from t to the next step s.

- `np.maximum(1 - ABAR[s] - sig ** 2, 0)` — rounding can make this a tiny negative number; clipped before the square root.
- `if s > 0:` — no fresh noise on the final jump to step 0: the result is the sample.

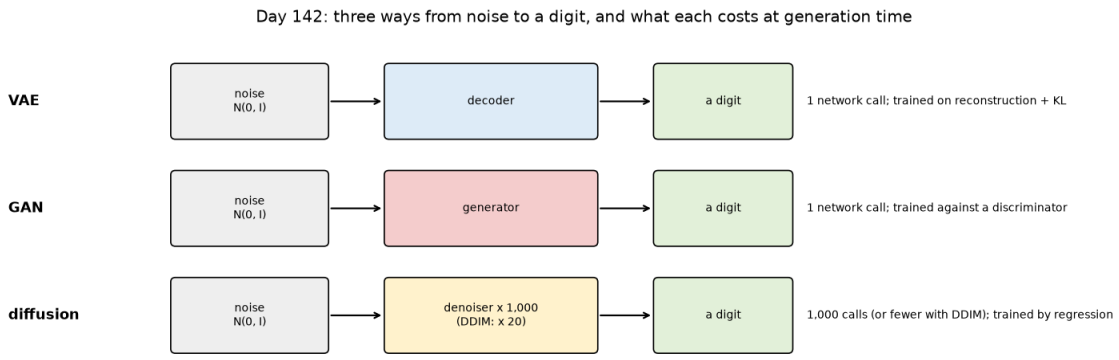
net_x0_fn, grey_share, n_params

CODE

```
def net_x0_fn(net):
    return lambda x, t: d141.guess(net, x, np.full(len(x), t))[2]
def grey_share(F, lo=0.25, hi=0.75):
    return float(((F > lo) & (F < hi)).mean())
def n_params(*nets):
    return int(sum(W.size + b.size for net in nets for W, b in net))
```

- `lambda x, t: d141.guess(net, x, np.full(len(x), t))[2]` — a one-line function that remembers `net`; np.full gives every row the same t, and [2] picks the x_0 guess.
- `((F > lo) & (F < hi)).mean()` — two boolean arrays combined with & (element-wise and); the mean of True/False is the share that is True.
- `def n_params(*nets):` — the star collects any number of arguments into a tuple: n_params(G, D) or n_params(net).
- `for net in nets for W, b in net` — two for clauses in one generator: every layer of every network.

SYSTEM DIAGRAM



The three families on one scorer.

Segment 2 — The experiments

demo_hand: the identity checks

CODE

```
f = lambda x, t: np.tanh(x)
A = d140.reverse_sample(lambda x, t: (x - np.sqrt(ABAR[t]) * f(x, t)) / np.sqrt(1 - ABAR[t]), 100,
    np.random.RandomState(4), "posterior", d=784)
B = ddim_sample(f, 100, np.random.RandomState(4), 1000, eta=1.0)
print(f"Day 140's sampler against DDIM with eta 1 and all 1,000 steps, same guesser, same random
    numbers: "
      f"largest difference {np.abs(A - B).max():.2e}")
B2 = ddim_sample(f, 100, np.random.RandomState(4), 20, eta=0.0); B3 = ddim_sample(f, 100,
    np.random.RandomState(9), 20, eta=0.0)
B4 = ddim_sample(f, 100, np.random.RandomState(4), 20, eta=0.0)
```

- `lambda x, t: np.tanh(x)` — any function of the right shape will do: the checks are about the samplers, not the guess.

- `lambda x, t: (x - np.sqrt(ABAR[t]) * f(x, t)) / np.sqrt(1 - ABAR[t])` — Day 140's sampler wants a noise guesser, so the x_0 guesser is wrapped: the noise its guess implies.
- `np.random.RandomState(4)` — two generators made with the same seed give the same numbers: both samplers see the same noise.

demo_vae and demo_gan: time only the training

CODE

```
def demo_vae(X, sc):
    t0 = time.perf_counter(); enc, dec, _ = d129.train_vae(X[:60000], 16, epochs=VAE_EPOCHS); took
        = time.perf_counter() - t0
    t0 = time.perf_counter(); F = d130.sample_prior(dec, 2000, 16,
        np.random.RandomState(1)).astype(np.float32); ts = time.perf_counter() - t0
    return dict(name="VAE", F=F, train=took, sample=ts, calls=1, scores=sc(F),
        params=n_params(enc, dec), gen=n_params(dec))
def demo_gan(X, sc):
    G = d126.init_net([64, 256, 784], 0); D = d126.init_net([784, 256, 1], 1)
    oG, oD = d134.AdamGAN(G, 2e-4, 0.5), d134.AdamGAN(D, 2e-4, 0.5); rng =
        np.random.RandomState(0)
    Z = np.random.RandomState(1).standard_normal((2000, 64)).astype(np.float32); log = []; took =
        0.0
    for ep in range(1, max(GAN_EPOCHS) + 1):
        t0 = time.perf_counter(); o = rng.permutation(60000)
        for s in range(0, 60000 - 127, 128):
            d134.gan_step(G, D, oG, oD, X[o[s:s + 128]], rng, 64)
        took += time.perf_counter() - t0
        if ep in GAN_EPOCHS:
            t0 = time.perf_counter(); F, _ = d134.generate(G, Z); ts = time.perf_counter() - t0
            F = F.astype(np.float32); log.append((ep, *sc(F)))
    return dict(name="GAN", F=F, train=took, sample=ts, calls=1, scores=sc(F), params=n_params(G,
        D), gen=n_params(G), log=log)
```

- `t0 = time.perf_counter(); ...; took = time.perf_counter() - t0` — a clock read before and after: the difference in seconds.
- `took += time.perf_counter() - t0` — the GAN's clock is added up epoch by epoch, stopped before scoring.
- `return dict(name="VAE", F=F, ...)` — dict(key=value, ...) builds a dictionary from keyword arguments: every model returns the same keys.

demo_diffusion: the averaged copy lives in a hook

CODE

```
def demo_diffusion(X2, sc):
    ema, log = [None], []; t_sample = [0.0]
    def hook(s, net):
        if ema[0] is None:
            ema[0] = [(W.copy(), b.copy()) for W, b in net]
        else:
            ema_update(ema[0], net)
        if s in CHECKS:
            for name, n in (("raw", net), ("averaged", ema[0])):
                F, took, r = d141.sample_scores(n, sc); t_sample[0] += took; log.append((s, name,
                    *r, grey_share(F)))
    t0 = time.perf_counter(); net, _ = d141.train_image_denoiser(X2[:60000], STEPS, hook=hook);
        total = time.perf_counter() - t0
    F, ts, r = d141.sample_scores(ema[0], sc)
    return dict(name="diffusion", F=F, train=total - t_sample[0], sample=ts, calls=1000, scores=r,
        params=n_params(net), gen=n_params(net),
                log=log, net=net, ema=ema[0])
```

- `ema, log = [None], []` — a one-item list holding None: the hook can replace ema[0] without a nonlocal statement.

- `if ema[0] is None:` — the first call copies the weights; every later call averages.
- `[(W.copy(), b.copy()) for W, b in net]` — real copies: without .copy() the averaged copy would be the training weights themselves.
- `train=total - t_sample[0]` — the checkpoint sampling time is subtracted, leaving training (and the averaging).

demo_ddim: fourteen runs from one model

CODE

```
f = net_x0_fn(diff["ema"]); out = []; keep = {}
for eta in (0.0, 1.0):
    for steps in (1000, 250, 100, 50, 20, 10, 5):
        t0 = time.perf_counter(); F = d141.to_pixels(ddim_sample(f, 2000,
            np.random.RandomState(1), steps, eta)); took = time.perf_counter() - t0
        i, fid, p, r, n = sc(F); out.append((steps, eta, fid, took)); keep[(steps, eta)] = F[:16]
        print(f"{steps:6d} {eta:4.0f} {i:7.3f} {fid:8.3f} {p:10.3f} {r:7.3f} {grey_share(F):11.4f}
            {took:6.1f}s")
```

- `for eta in (0.0, 1.0): for steps in (...):` — two nested loops: 2 × 7 = 14 runs.
- `keep[(steps, eta)] = F[:16]` — 16 samples of each run, under a tuple key, for the figure.
- `i, fid, p, r, n = sc(F)` — the scorer's five results unpacked into five names.

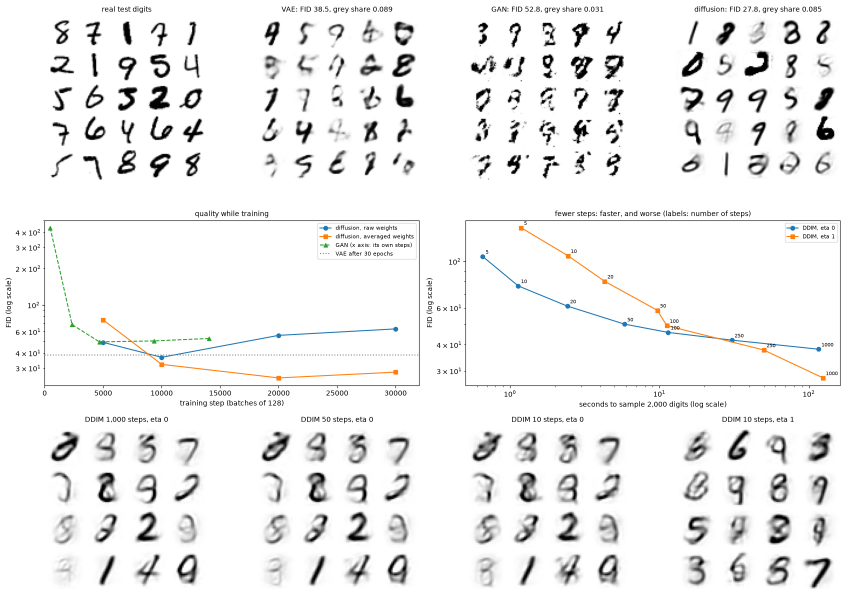
demo_table: picking one row with next

CODE

```
best20 = next(r for r in ddim if r[0] == 20 and r[1] == 0.0)
rows = [("VAE", vae["train"], vae["sample"], 1, vae["scores"], grey_share(vae["F"]),
    vae["params"], vae["gen"]),
        ("GAN", gan["train"], gan["sample"], 1, gan["scores"], grey_share(gan["F"]),
            gan["params"], gan["gen"]),
        ("diffusion, 1,000 steps", diff["train"], diff["sample"], 1000, diff["scores"],
            grey_share(diff["F"]), diff["params"], diff["gen"])]
print(f"{'model':>22} {'train':>7} {'sample 2,000':>13} {'net calls':>10} {'FID':>8} {'recall':>7}
    {'grey':>7} {'numbers':>9} {'sampler':>9}")
for name, tr, ts, calls, s, g, p, gp in rows:
    print(f"{name:>22} {tr:6.0f}s {ts:12.2f}s {calls:10,d} {s[1]:8.3f} {s[3]:7.3f} {g:7.4f}
        {p:9,d} {gp:9,d}")
```

- `next(r for r in ddim if ...)` — the first item of a generator that matches: one row, without a loop.
- `{calls:10,d}` — a format spec: width 10, with thousands separators.
- `for name, tr, ts, calls, s, g, p, gp in rows:` — eight names unpacked from each 8-item tuple.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `os.environ.setdefault(k, v)` — set an environment variable unless already set.
- `a *= b (numpy)` — changes the array in place; a = a * b makes a new one.
- `zip(a[:-1], a[1:])` — neighbouring pairs of a sequence.
- `def f(*args)` — any number of positional arguments, as a tuple.
- `next(x for x in it if ...)` — the first match.
- `dict(k=v, ...)` — a dictionary from keyword arguments.

End of Day 142 syntax reference. For the experiments and what they showed, see the Day 142 complete guide.